40. Observability: Logs, Metrics, Tracing

40. Observability: Logs, Metrics, Tracing

Phase: Phase 6 — Systems and infrastructure
Estimated time: 1–2 days
Slug: `40-observability`

Learning objectives

- Emit structured logs
- Define RED/USE metrics
- Propagate trace IDs
- Make dashboards actionable

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Three pillars

- **Logs**: discrete events (structured JSON)
- **Metrics**: aggregates (counters, histograms)
- **Traces**: request journeys across services

```
slog.Info("request completed", "method", r.Method, "path", r.URL.Path, "status",
status, "dur_ms", ms)
```

Metrics that matter

For services: rate, errors, duration (RED). Histograms for latency percentiles.

Tracing

Pass trace context in headers; OpenTelemetry is the common standard. Sample wisely in production.

Common mistakes (especially from Python instincts)

- Println debugging as the only strategy
- High-cardinality metric labels (user IDs)
- Logging secrets/PII

Exercises

Work in `exercises/40-observability/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/40-observability/` until you have an honest failing attempt.

Self-check

1. RED metrics are?
2. Why structured logs?
3. Danger of high cardinality?

Further reading

- https://pkg.go.dev/log/slog
- https://opentelemetry.io/